IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Build Any Robust Multidisciplinary Product with AI Agents

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that span software and hardware – a web application, firmware, a circuit board and a printed case – with AI coding agents. It gives every product the same structure: each fact is declared once in typed files, every artifact that depends on it is derived by one command, and checks on every commit refuse anything that no longer follows from its declaration. Hard problems such as part placement and solid modelling are handed to established engines, so the platform grows by declarations rather than by special cases. To test it, 20 realistic mistakes were injected into two products. The first run caught 15. Each miss was turned into a general rule, and the current version catches 20 of the 20 while letting every correct change through.**

Keywords—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration

I. Introduction

While building Outreach [1], an off-grid messaging system made of radios, gateways, a companion app and printed cases, the author ran into the same problem more than once. The firmware and the app disagreed on the layout of a message, and a printed case did not fit the board it was made for. Neither part was wrong on its own. The facts they shared were copied by hand between different tools, one copy changed, and the mistake showed up only when the parts were put together.

This is how most products are built, and AI coding agents [2], [3] make it faster without making it safer. An agent is quick inside one codebase but knows nothing about the others: the agent writing the firmware does not know the board changed, and the one writing the web page does not know the firmware did. Every project also invents its own layout, scripts and habits, so each new product starts from scratch and every agent has to learn it again.

Fiducial [4] takes the opposite approach. Every product is built the same way, in one repository, around one rule: each fact is declared once, and everything that needs it is derived from it and checked (Fig. 1). People and agents edit the declarations, never the outputs. This paper explains the idea, how the platform is put together and why, and how its checks hold up against mistakes made on purpose.

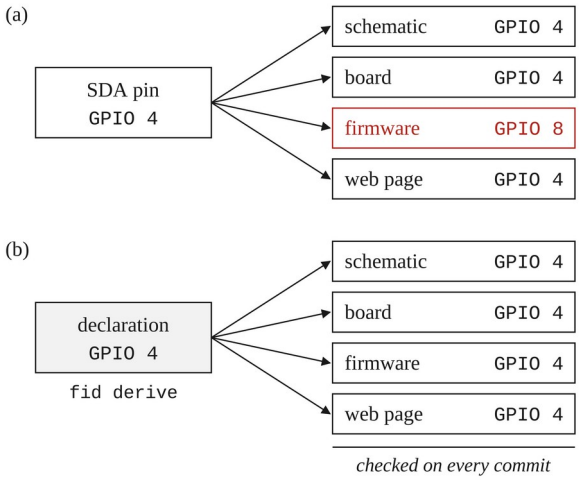


Fig. 1. One fact in four places. (a) Copied by hand: one copy changes and nothing notices. (b) Declared once: every copy is derived from it and checked on every commit.

II. The Idea

A product is a set of facts – which pin a sensor is on, what a button says in each language, the brand colour, the size of the board – and a set of artifacts that use them: code, schematics, translations, web pages, case models. Most bugs between disciplines come from one fact being written in more than one artifact. Fiducial removes that with three rules.

Declare once. Every fact has exactly one place where it is written, a typed text file that a person or an agent edits.

Derive everything else. One deterministic command, fid derive, generates every artifact that depends on a fact. Generated files are never edited by hand.

Check every change. On every commit, checks confirm that every artifact still follows from its declarations and that the declarations make sense. A failure names the declaration at fault.

None of these rules is new on its own. Model-driven engineering generates code from models [5], and Nix identifies every build by a hash of its inputs [6]. What Fiducial adds is applying them across every discipline of one product at once, in plain files that coding agents read and write well, with the checks – not the agent's good behaviour – as what makes the result reliable.

III. How It Is Built

A. One Structure for Every Product

fid new creates a product repository with a fixed layout, and fid add turns on capabilities: packaged pipelines for web frameworks, translations, content, search metadata, legal pages, design systems, databases, deployment, device messages, firmware for two microcontroller families, and hardware – 23 in all. Two products built with Fiducial look the same, so knowing one means knowing the others, and a new product starts from working, checked parts instead of an empty folder. Every product also carries an AGENTS.md file [7] and one instruction file per capability, so any agent starts from the same context. A newer platform version comes in with one command, fid upgrade, which keeps the product's own edits.

B. Declarations

Declarations are typed text files in version control: messages/<language>.json for every user-visible string, fiducial.toml for pages and brand, design-system.md for colours and the contrast they must reach, protocol.toml for the messages a device sends, and hardware/product.toml for the parts of a physical product and how they connect. Plain text was chosen over a modelling tool because it is what coding agents handle best, and because every change then goes through a normal code review. A declaration states a fact and nothing more. It does not say where the sitemap lists a page or which way a trace runs.

C. Derivation

fid derive runs every enabled pipeline. Each one reads the declarations it needs and writes its artifacts: a typed message module, a sitemap, a database migration, a firmware pin map, TypeScript types, a circuit board. Every pipeline lists the files it reads and writes, and the SHA-256 hash of each is stored in a lock file. Derivation is deterministic, so the same inputs always give the same bytes; without that, comparing hashes would mean nothing. This requirement decided more than one design choice: the placement solver, for example, runs on a single thread, because with several it gave different answers to the same problem.

D. Checks

The checks are of three kinds (Fig. 2). Freshness: fid derive --check fails when a declaration changed without its outputs being regenerated, or when an output was edited by hand. Rules inside derivation: every language must have every string, every page needs a title, every colour pair must reach its contrast, circuit voltages must fall inside their windows, the parts must cost less than a declared limit, and the board must fit its case. Consumers: compilers and type checkers read the derived files, so code that still uses a renamed fact stops compiling. All of them run in continuous integration on every commit. A failure names the file and the declaration at fault, which is the kind of signal an agent needs to fix its own mistake.

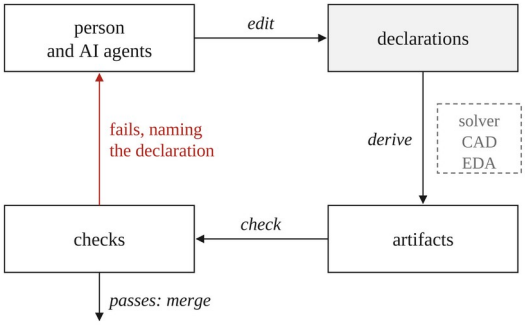


Fig. 2. How a change moves through Fiducial. Declarations are edited, derivation produces the artifacts and hands hard problems to established engines, and the checks either let the change through or return a message naming the declaration at fault.

E. Hard Problems Go to Engines

Some derivations are real engineering problems: where to put the parts on a board, how to route it, how to build a case around it. The first version placed parts with a hand-written search, and every new product added another special rule. The rules began to interact, and the code grew faster than the products. So the structure changed: the platform writes the problem down in a fixed format and hands it to an established engine – the CP-SAT constraint solver [8] for placement, the OpenCascade kernel through build123d [9] for the case, and KiCad [10] for the board and its design-rule check. Every placement rule became one of eight generic kinds, and a new requirement became a new declaration instead of new code. When declarations contradict each other, the solver names the smallest set that cannot all hold.

F. Mistakes Become Rules

Two habits keep the platform growing in the right direction. Agents fix a problem in the layer that owns the fact – a wrong pin in the declaration, a wrong derivation in the platform – and never in a generated file. And when a mistake gets past the checks, it is not patched for that one case; it becomes a general rule that holds for every product. Section V shows what that looks like.

IV. Example: A Sensor Stick

To show the whole chain on something anyone can inspect, the repository contains a public example: a USB-C stick that reports temperature and humidity to a

web page (Fig. 3). It is small on purpose, but it has every discipline: a printed case, a routed two-layer board with 23 parts, Rust firmware and a web page. The person writes the product declaration, an outline drawing, the firmware and the page. Fiducial derives the rest, including the files a board house needs to make it.

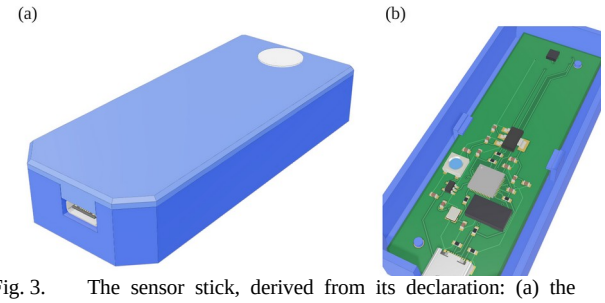


Fig. 3. The sensor stick, derived from its declaration: (a) the closed case, 36 × 84 mm; (b) the routed board inside it, with the sensor at the far end under its vent.

The message the stick sends is declared once as well, and both the firmware's encoder and the page's decoder are generated from it. That closes the exact gap met in Outreach, where the two sides were written separately. Moving the sensor's data line to another pin is one changed line. Until the product is re-derived, the check fails; after fid derive, the board, the schematic, the firmware's pin map and the page's types all follow, and the firmware and the page need no edit. If the new pin is one the chip's I²C bus cannot use, the firmware stops compiling, because in Embassy [11] every pin is its own type.

V. Evaluation

A. Development

Fiducial was built over six weeks, from 6 September to 15 October 2026, by one person working with AI agents: about 240 commits and 68 merged pull requests, with about 65,000 lines of Rust, 9,000 of TypeScript and 6,000 of Python. It has been used on two products, the sensor stick and a private one. Table I lists measurements from both.

TABLE I. Measured Results

Measure	Result
Placement rule kinds	8; none added for either product
Sensor stick board	26 × 79 mm, 2 layers, 0 DRC errors
Sensor stick parts	USD 4.74 per board at 5 (limit USD 15)
One pin moved	8 generated files rewritten in 37 s
Placement, private product	13 % shorter connections than the hand-written search
Contradictions	named as the smallest conflicting set

B. Injected Mistakes

Checks are only worth having if they catch the mistakes people actually make. To measure that, the test follows the idea of mutation testing [12]: inject a realistic mistake, run the checks, and see whether they notice. Twenty mistakes were chosen to cover the eight kinds of drift met while building Fiducial and its products (Table II): ten in a fresh web product in English and Serbian with ten software capabilities, and ten in the sensor stick's hardware, firmware and interface. Each is applied to a clean copy, the checks run in the same order as in CI – derive, --check, the firmware build and tests, the page's test and type check – and the copy is reset. Three correct changes and two runs with no change test the other direction: that the checks do not fail without a reason. The test is a script in the repository and runs with one command.

On release 0.9.4 the checks caught 15 of the 20. The five misses were not random: each was a fact the platform did not yet own. Two declarations edited without re-deriving went unnoticed, because only the hardware pipeline listed the files it read. Firmware that named a pin by its number kept compiling after the pin moved. A mistyped sensor address existed only in the firmware. A Serbian string left in English passed every structural check. And the firmware and the page disagreeing on a message layout was caught only by a hand-written test.

TABLE II. Injected Mistakes, by Kind of Drift

Mistake	0.9.4	Now
Stale output		
English string edited, not re-derived	not caught	--check
Domain changed, not re-derived	not caught	--check
Sensor pin moved, not re-derived	--check	--check
Generated file edited		
Message module edited	--check	--check
Worker config edited	--check	--check
Applied migration edited	--check	--check
Firmware pin map edited	--check	--check
Message layout changed in firmware	test only	--check
Incomplete declaration		
String added in English only	derive	derive
Post published in English only	derive	derive
Page without a title	derive	derive
Declaration breaks a rule		
Text below 4.5 : 1 contrast	derive	derive
USB-C pull-down 5.1 → 22 kΩ	derive	derive
Cost limit lowered to USD 3	derive	derive
Declarations conflict		
Sensor zone contradicts its vent	derive	derive
Case narrowed 36 → 28 mm	derive	derive
Code out of date		
Net renamed, code not updated	compiler	compiler
Declaration bypassed		
Pin named by number, then moved	not caught	derive
Sensor address typed in firmware	not caught	derive
Copied, not translated		
Serbian string left in English	not caught	--check

Each miss became a general rule: every pipeline lists its inputs; firmware that names a pin or an I²C address directly is refused; the sensor's address is declared with the part and derived into the firmware; a translation identical to the default language fails unless

it is declared the same on purpose; and every message is declared once, with both sides' code generated. On the current version, all twenty mistakes are caught: ten during derivation, nine by --check and one by the compiler. Every correct change passes and the runs with no change leave every file as it was. Every failure names the file and the declaration at fault (Fig. 4).

(a)

```
$ fid derive
fid-i18n: the catalogs are not complete.
messages/sr.json is missing 1 key(s):
action.delete
```

(b)

```
$ fid derive
firmware types what the board already declares (a pin,
an address, a command or a wait):
firmware/rp2040/src/main.rs:84: takes PIN_4 directly —
take it through `board::<net>!(p)` …
```

Fig. 4. Two failures from the test, as the person or agent sees them: (a) a string added in English only; (b) firmware that names a pin directly instead of taking it from the declaration.

The twenty were written by the author, who also wrote the platform, so they may favour what it was built to catch. Later rounds of mistakes, written separately from the platform's code, found further gaps – among them a unit mismatch between firmware and page, an I²C line with no pull-up resistor, and cookie consent that started with a choice already made – and each was again turned into a rule.

VI. Discussion

Better models move the line between what a person must check and what can be left to an agent, but they do not remove it. A probabilistic system cannot be guaranteed to be correct, and even a deterministic tool is only right about what it was told. Fiducial bounds that uncertainty: declarations are the constraints, derivation is repeatable, and the checks refuse drift. The person still judges what no check can. During this work that meant a parts catalogue that changed format overnight, a text match that read "±0.1 %" as "1 %", and a sensor that had to be moved away from the board's heat.

The main lesson is about structure more than any one tool. Keeping every discipline in one repository with one shape made the product something an agent can see whole, and turning each escaped mistake into a rule made the checks stronger with use.

The evidence has clear limits. Only two products were built, one of them public, and neither has been manufactured and tested yet. The test measures the checks, not whether agents working with Fiducial make fewer mistakes than agents working without it. Circuit checks are DC only, and autorouting is not deterministic, so a routed board is checked rather than compared byte for byte.

VII. Conclusion

Fiducial gives a product one structure across software and hardware: each fact declared once, everything else derived, and every change checked. That makes development quicker, because a new product starts from working parts and a change is made in one place, and more reliable, because a mistake is caught with a message that names its cause. Twenty injected mistakes found five gaps in the first run; once each became a general rule, all twenty are caught. The next steps are to manufacture the sensor stick and to compare agents working with and without the platform on the same tasks. The platform and the example are open source [4].

VIII. References

[1] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[2] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[3] OpenAI Codex. Available at: https://openai.com/codex/

[4] A. Zdravković, Fiducial, release 0.9.11, 2026. Available at: https://github.com/AleksaZCodes/fiducial

[5] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006.

[6] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[7] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[8] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[9] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[10] KiCad EDA. Available at: https://www.kicad.org/

[11] Embassy, an async framework for embedded Rust. Available at: https://embassy.dev/

[12] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978.